ARYA COLLEGE OF ENGINEERING & IT

Jaipur, Rajasthan, India

Department of Computer Science and Engineering

SOFTWARE REQUIREMENTS SPECIFICATION

For

SKYTECH AIRLINES

Airline Reservation System

Submitted in partial fulfilment of the requirements for the Bachelor of Technology degree in Computer Science and Engineering.

Academic Session: 2026–2027 | Semester: V

Project Period: 6 July 2026 – 6 October 2026

Project Mentor: Er. Ram Babu Buri

Team Name: SkyTech

Team Leader: Mohammed Sahil

Team Members

1. Mohammed Sahil (Team Leader) — 24EARCS095
2. Mukesh Sah — 24EARCS097
3. Muskan Raj — 24EARCS098
4. Nikhil Ranjan Choubey —24EARCS103
5. Prashant Chaudhary — 24EARCS115

Project and Team Information

Table 1. Project team

S. No.	Team Member Name	Role	Assigned Module	RTU Roll Number
1	Mohammed Sahil	Team Leader	Module 1: Authentication & User Management	24EARCS095
2	Nikhil Ranjan Choubey	Team Member	Module 2: Flight Search & Availability	24EARCS103
3	Mukesh Sah	Team Member	Module 3: Booking & Passenger Management	24EARCS097
4	Muskan Raj	Team Member	Module 4: Ticketing & Cancellation	24EARCS098
5	Prashant Chaudhary	Team Member	Module 5: Admin Dashboard & Flight Management	24EARCS115

Project Supervision

Table 2. Project supervision details

Item	Detail
Project Mentor	Er. Ram Babu Buri
Department	Computer Science and Engineering
Institution	Arya College of Engineering & IT, Jaipur, Rajasthan, India
University	Rajasthan Technical University, Kota, Rajasthan
Programme / Semester	B.Tech, Computer Science and Engineering, 5th Semester
Academic Session	2026–2027
Project Period	6 July 2026 – 6 October 2026
Team Name	SkyTech

Declaration

This project is an academic team effort. The team members are individually responsible for the module assigned to them and collectively responsible for the coordinated integration, testing, and documentation of the SkyTech Airlines application. This document is a draft prepared for review; it carries no mentor signature, institutional approval, or registration number, and none should be inferred.

Document Control and Version History

Table 3. Document control

Attribute	Value
Document title	Software Requirements Specification
Project name	SkyTech Airlines – Airline Reservation and Passenger Management System
Document identifier	STAIR-SRS-001
Version	1.0
Status	Draft for review
Prepared by	Project team (Team Leader: Mohammed Sahil)
Reviewed by	Project mentor, Er. Ram Babu Buri — review pending
Creation date	13 October 2026
Last revision date	13 October 2026
Academic session	2026–2027

Abstract

Airline reservation involves several interdependent activities: discovering flights, recording passenger information, issuing booking references, producing tickets, handling cancellations, and maintaining flight schedules. When these activities are performed manually or through poorly coordinated tools, records become inconsistent, booking references are hard to trace, and administrators lack a reliable view of operations. SkyTech Airlines – Airline Reservation and Passenger Management System is proposed as a centralized web application that addresses these problems within an academic scope.

The system recognizes two user categories. Passengers register, authenticate, search flights by origin, destination, and date, review schedules, fares, and database-derived seat availability, enter passenger details, and create bookings that receive a unique PNR. They may then view tickets and booking history, retrieve a booking by PNR, and request cancellation, which updates booking status. Administrators access a protected dashboard to manage flights, routes, schedules, and fares and to review booking records and reports. Role-based access control is enforced on the server.

The proposed architecture uses Java 21, JavaServer Pages, Jakarta Servlets, JDBC, and a MySQL relational database, packaged with Apache Maven and deployed on Apache Tomcat 10.1. Presentation, request processing, business logic, and data access are separated to support maintainability, and the work is divided into five modules assigned to individual team members. The expected benefits are centralized records, reduced manual effort, improved consistency of booking information, and easier booking management.

This SRS describes planned capabilities and requirements. It does not assert that any feature has been implemented or tested, and it excludes real-time airline integration, actual payment processing, live seat inventory, email delivery, and production deployment unless these are later implemented and verified.

Keywords: Software Requirements Specification, Airline Reservation, Java Web Application, JSP, Jakarta Servlets, JDBC, MySQL, Role-Based Access Control, PNR.

Table of Contents

Project and Team Information 2
Project Supervision 2
Declaration 2
Document Control and Version History 3
Abstract 3
List of Abbreviations 8
1. Introduction 10
1.1 Purpose 10
1.2 Project Scope 10
1.3 Problem Statement 11
1.4 Project Objectives 11
1.5 Intended Audience 12
1.6 Definitions and Abbreviations 12
1.7 References and Standards 13
2. Overall Description 13
2.1 Product Perspective 13
2.2 Product Functions 14
2.3 User Classes and Characteristics 14
2.4 Operating Environment 15
2.5 Design and Implementation Constraints 15
2.6 User Documentation 16
2.7 Assumptions and Dependencies 16
3. System Requirements 16
3.1 Hardware Requirements 16
3.2 Software Requirements 16
3.3 External Interface Requirements 17
4. Functional Requirements 18
4.1 Module 1 — Authentication and User Management 19
4.2 Module 2 — Flight Search and Availability 20
4.3 Module 3 — Booking and Passenger Management 21
4.4 Module 4 — Ticketing and Cancellation 23
4.5 Module 5 — Admin Dashboard and Flight Management 25
5. Non-Functional Requirements 26
5.1 Security 26
5.2 Performance 27
5.3 Reliability and Data Integrity 27
5.4 Usability 28
5.5 Maintainability 28
5.6 Compatibility 28
5.7 Scalability 28

5.8 Privacy 28
5.9 Availability and Recovery 28
5.10 Accessibility 29
6. System Architecture and UML Diagrams 29
7. Database Requirements and Data Dictionary 37
7.1 Users Table 37
7.2 Flights Table 37
7.3 Bookings Table 37
7.4 Passengers Table 37
7.5 Tickets Table 37
7.6 Data Dictionary 38
7.7 Database Integrity Rules 39
8. Team Responsibilities and Module Integration 40
8.1 Integration Dependencies 40
8.2 Shared Development Responsibilities 41
9. Verification, Validation and Test Cases 41
9.1 Security Testing 44
9.2 Acceptance Testing 45
10. Acceptance Criteria 45
11. Assumptions, Dependencies and Constraints 46
11.1 Assumptions 46
11.2 Dependencies 46
11.3 Constraints 47
12. Future Enhancements 47
13. Conclusion 48
14. References 48
Items to Confirm Before Submission 49

List of Figures

Figure 1. High-Level System Architecture (proposed) 30
Figure 2. UML Use-Case Diagram (proposed) 31
Figure 3. Passenger Login Sequence Diagram (proposed) 32
Figure 4. Flight Search and Booking Sequence Diagram (proposed) 33
Figure 5. Conceptual Class Diagram (proposed) 34
Figure 6. Ticket Cancellation Activity Diagram (proposed) 35
Figure 7. Entity-Relationship Diagram (proposed conceptual) 36

List of Tables

Table 1. Project team 2
Table 2. Project supervision details 2
Table 3. Document control 3
Table 4. Revision history 3
Table 5. Abbreviations used in this document 8
Table 6. Intended audience and relevance 12
Table 7. Definitions of key terms 12
Table 8. Responsibilities of each architectural layer 13
Table 9. Module overview: inputs, processing, outputs, and database interaction 14
Table 10. Proposed operating environment 15
Table 11. Recommended development hardware 16
Table 12. Software requirements and verification status 16
Table 13. User-interface pages 17
Table 14. Module 1 functional requirements 19
Table 15. Module 2 functional requirements 20
Table 16. Module 3 functional requirements 22
Table 17. Module 4 functional requirements 23
Table 18. Module 5 functional requirements 25
Table 19. Proposed data dictionary 38
Table 20. Module responsibilities 40
Table 21. Shared responsibilities 41
Table 22. Functional test cases (all unexecuted) 42
Table 23. Security test cases (all unexecuted) 44
Table 24. Requirements-to-acceptance matrix 45
Table 25. Possible future enhancements 47

List of Abbreviations

Table 5. Abbreviations used in this document

Abbreviation	Expansion
ACEIT	Arya College of Engineering & IT
ADMIN / PASSENGER	Role values used for authorization
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
DAO	Data Access Object
DBMS	Database Management System
ER	Entity-Relationship

Abbreviation	Expansion
FR / NFR	Functional Requirement / Non-Functional Requirement
HTTP / HTTPS	Hypertext Transfer Protocol / Hypertext Transfer Protocol Secure
JDBC	Java Database Connectivity
JSP	JavaServer Pages
MVC	Model-View-Controller
PNR	Passenger Name Record (booking reference)
RBAC	Role-Based Access Control
RTU	Rajasthan Technical University
SQL	Structured Query Language
SRS	Software Requirements Specification
UI / UX	User Interface / User Experience
UML	Unified Modeling Language
WAR	Web Application Archive
XSS	Cross-Site Scripting

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional requirements, non-functional requirements, external interfaces, data requirements, design constraints, and acceptance criteria for SkyTech Airlines – Airline Reservation and Passenger Management System. It is the shared reference against which the system is designed, built, tested, and evaluated.

The document is intended to guide the following activities:

- **Development:** it states what each part of the system shall do, so that implementation decisions can be traced to a requirement.
- **Module-level implementation:** it assigns every functional requirement to one of five modules and one module owner.
- **Team collaboration:** it fixes common terminology, status values, and module interfaces for the five developers.
- **Database design:** it proposes the entities, relationships, and integrity rules that the schema must support.
- **Integration:** it describes the dependencies between modules and the order in which they must interoperate.
- **Testing:** every requirement carries an acceptance condition, and Section 9 derives test cases from them.
- **Maintenance:** it records design intent for future maintainers.
- **Academic evaluation:** it provides evaluators and the project mentor with a verifiable statement of scope and intent.

1.2 Project Scope

SkyTech Airlines is an academic web application intended to centralize the principal activities of airline reservation. The intended scope comprises:

- Passenger registration and login, with role-based access and profile management.
- Flight search by origin, destination, and travel date, with display of schedules, fares, and availability.
- Passenger-detail entry, flight booking, PNR generation, seat allocation, and booking confirmation.
- Ticket generation, booking-history management, PNR-based lookup, and ticket cancellation.
- Administrative flight management, including routes, schedules, and fares.

- An administrative dashboard and reports derived from stored data.

Scope boundary. The system is an academic reservation application and is not a production airline reservation platform. It is not assumed to be connected to actual airline inventory, a global distribution system, a payment gateway, government identity services, or external airline APIs. Seat availability reflects the application's own database only. Payment processing, refund handling, email or SMS delivery, and regulatory certification are outside the baseline scope unless separately specified, implemented, and verified.

1.3 Problem Statement

Manual or poorly coordinated reservation processes exhibit recurring limitations:

- Passenger records are difficult to keep consistent, and data may be duplicated across registers or files.
- Searching for suitable flights is time-consuming when schedules and fares are not held centrally.
- Manual entry introduces errors in booking information.
- Booking references are difficult to track and retrieve.
- Ticket and cancellation records may disagree with one another.
- Administrators have limited visibility of flights, bookings, and cancellations.
- Access control is not centralized, so sensitive data may be seen by unintended users.

The proposed application addresses these limitations by holding users, flights, bookings, passengers, and tickets in a single relational database; exposing them through role-restricted web pages; generating unique booking references; and applying validation and transactional rules to reservation changes.

1.4 Project Objectives

The project shall be considered to have met its objectives when each of the following can be demonstrated by test evidence:

1. Provide secure user authentication (FR-01 to FR-08).
2. Allow passengers to search for flights by origin, destination, and date (FR-09, FR-13).
3. Display flight schedules, fares, and availability information (FR-10, FR-12).
4. Maintain passenger and booking records (FR-15, FR-16, FR-20).
5. Generate a unique booking reference for every successful booking (FR-17).
6. Support ticket retrieval and booking-history access (FR-24 to FR-27).
7. Record cancellation requests and update booking status (FR-28 to FR-32).

8. Provide administrators with controlled flight-management capabilities (FR-34 to FR-42).
9. Maintain database integrity through constraints and transactions (Sections 5.3 and 7.7).
10. Support testing and future extension through modular design (Sections 5.5 and 8).

1.5 Intended Audience

Table 6. Intended audience and relevance

Reader	Relevance of this document
Project mentor	Reviews scope, requirements, and acceptance criteria; confirms the design before implementation is evaluated.
Student development team	Uses it as the common reference for module boundaries, interfaces, and terminology.
Software developers	Implement the functional and non-functional requirements of their assigned modules.
Database designers	Reconcile the proposed entities and integrity rules with the actual schema.
Testers	Derive test cases and verify acceptance conditions against the requirements.
Academic evaluators	Assess completeness, technical soundness, and consistency of the specification.
Future maintainers	Understand design intent, constraints, and extension points.

1.6 Definitions and Abbreviations

Table 7. Definitions of key terms

Term	Definition
SRS	Software Requirements Specification: a document that describes what a software system shall do.
RTU	Rajasthan Technical University, Kota, to which the college is affiliated.
PNR	Passenger Name Record: in this project, the unique booking reference used to retrieve a reservation. The meaning follows the project's own terminology and is not linked to any airline reservation system.
UI / UX	User Interface / User Experience: the visible controls of the system and the quality of the interaction.
DBMS	Database Management System: software that stores and manages data (MySQL in this project).
JDBC	Java Database Connectivity: the Java API for executing SQL against a relational database.
JSP	JavaServer Pages (Jakarta Pages): server-side pages that generate HTML for the browser.
HTTP / HTTPS	Hypertext Transfer Protocol and its encrypted variant, used between browser and server.
RBAC	Role-Based Access Control: permissions granted according to a user's role (PASSENGER or ADMIN).
UML	Unified Modeling Language: a standard notation for modelling software.

Term	Definition
CRUD	Create, Read, Update, Delete: the four basic data-management operations.
SQL	Structured Query Language: the language used to query and modify relational data.
MVC	Model-View-Controller: a pattern separating data, presentation, and request handling.
WAR	Web Application Archive: the packaged form of a Java web application deployed to a servlet container.

1.7 References and Standards

This SRS is structured according to the principles of a formal requirements specification and is informed by recognized requirements-engineering practice: unique requirement identifiers, mandatory requirements expressed with "shall", testable acceptance conditions, and explicit separation of functional and non-functional requirements. It has not been checked against a specific edition of the IEEE standard for software requirements specifications, and no claim of formal compliance is made. The technical sources consulted are listed in Section 14.

2. Overall Description

2.1 Product Perspective

SkyTech Airlines is proposed as a self-contained, database-driven Java web application. Passengers and administrators use a standards-compliant web browser; a server-side application running in a servlet container processes each request; and a MySQL relational database stores all persistent data. The system does not depend on any external reservation, payment, or identity service.

Table 8. Responsibilities of each architectural layer

Layer	Responsibility
Presentation layer	JSP pages with HTML, CSS, and JavaScript render forms and results, display validation and status messages, and encode all dynamic output. They contain no database access.
Application and business-logic layer	Jakarta Servlets (and optional filters) receive requests, enforce authentication and authorization, validate input, and call service classes that apply reservation rules such as availability checks, PNR generation, and cancellation eligibility.
Data-access layer	DAO classes encapsulate JDBC access, use parameterized statements, and manage transaction boundaries when several related records change together.
MySQL database	Stores users, flights, bookings, passengers, and tickets and enforces primary keys, foreign keys, uniqueness, and check constraints.

The five modules interact as follows: Module 1 authenticates users and supplies the role used by every other module; Module 2 supplies flight information; Module 3 creates bookings from an authenticated user and a selected flight; Module 4 operates on the resulting booking and ticket records; and Module 5 maintains the flight data that Module 2 and Module 3 consume and reviews booking data produced by Module 3 and Module 4. If the implemented code uses a different structure, this section shall be revised to describe the implemented architecture.

2.2 Product Functions

Table 9. Module overview: inputs, processing, outputs, and database interaction

Module	Primary input	Processing	Primary output	Database interaction
1. Authentication & User Management	Registration data; credentials; profile edits	Validation; salted password hashing; credential verification; session and role handling	Account; authenticated session; profile view	Reads and writes Users
2. Flight Search & Availability	Origin, destination, travel date	Input validation; query of active flights; availability calculation; optional sort/filter	List of matching flights or a no-result message	Reads Flights (and Bookings for availability)
3. Booking & Passenger Management	Selected flight; passenger details	Revalidation; availability recheck; seat allocation; PNR generation; transactional save	Booking confirmation with PNR	Writes Bookings, Passengers; updates availability
4. Ticketing & Cancellation	PNR or history request; cancellation request	Ownership and eligibility checks; ticket display; status transition	Ticket view; history list; cancellation result	Reads Bookings, Tickets, Passengers; updates statuses
5. Admin Dashboard & Flight Management	Flight, route, schedule, and fare data	Authorization; validation; CRUD or deactivation; aggregation for reports	Dashboard, flight lists, reports	Reads and writes Flights; reads Bookings

2.3 User Classes and Characteristics

Passenger

A passenger is the primary end user and is not assumed to have technical expertise. A passenger may register an account; log in and log out; manage profile information; search for flights; view available flights; enter passenger details; make a booking; retrieve booking details; view tickets; view booking history; and request cancellation of their own booking. A passenger shall not be able to access administrative operations or another passenger's booking data.

Administrator

An administrator is a trusted, provisioned user responsible for operational data. An administrator may access the protected administrative dashboard; add flights; edit flight details; deactivate or remove flights according to business rules; manage routes, schedules, and fares; review booking records; and view administrative reports. Administrative privileges shall be enforced on the server side and shall not rely on hiding interface elements.

2.4 Operating Environment

The following is the proposed environment. It is distinct from the verified local development environment, which the team shall record separately once confirmed.

Table 10. Proposed operating environment

Component	Proposed environment
Operating system	Windows or another compatible operating system
Programming language	Java 21
Web technologies	JSP and Jakarta Servlets
Database	MySQL 8.0 or a compatible verified version
Database connectivity	JDBC
Build system	Apache Maven
Web container	Apache Tomcat 10.1
Browser	Modern standards-compliant web browser
Version control	Git and GitHub

Bootstrap 5 is not listed as part of the environment because its use has not been confirmed; it shall be added to this document only if it is configured in the project.

2.5 Design and Implementation Constraints

- The application shall be compatible with Java 21.
- The application shall use the Jakarta Servlet API 6.0, which is the version aligned with Apache Tomcat 10.1.
- The application shall be deployable to Apache Tomcat 10.1; Maven WAR packaging is assumed and shall be confirmed.
- Persistent data shall be stored in a MySQL relational database accessed through JDBC.
- Authentication shall be session-based, if confirmed in the implementation.
- All input shall be validated on the server side.
- Operations that change several related records shall use database transactions.

- Authorization shall be role-based and enforced on the server.
- Passwords shall be stored using a salted password-hashing algorithm.
- Development shall be team-based, with source code integrated through Git and GitHub.

All APIs, external integrations, and libraries named in this document shall be verified against the dependencies declared in the project build file before any statement about them is treated as a description of the implemented system.

2.6 User Documentation

The following documentation is expected to accompany the application: a passenger registration and login guide; a flight-search guide; a booking and ticket-retrieval guide; a cancellation guide; an administrator flight-management guide; and a deployment and setup guide covering database creation, configuration of connection settings, and deployment to Tomcat. These documents are deliverables to be produced and are not yet written.

2.7 Assumptions and Dependencies

- Users have access to a compatible web browser.
- The database server is available when the application is running.
- Administrators enter valid flight information.
- The application has a configured database connection.
- Passenger-provided information is subject to validation.
- Flight availability reflects the application's own database unless an external inventory service is implemented.

3. System Requirements

3.1 Hardware Requirements

The values below are recommendations for development and testing. They are not measured production requirements.

Table 11. Recommended development hardware

Component	Recommended configuration
Processor	Intel Core i3 (or equivalent) or higher
RAM	8 GB or more
Storage	At least 5 GB free for tools, dependencies, and the database
Display	1366 × 768 or higher
Network	Internet access for dependency download and repository synchronization

3.2 Software Requirements

No component below is claimed to be installed or verified; versions shall be recorded from the project build file and the actual environment.

Table 12. Software requirements and verification status

Component	Proposed technology	Purpose	Verification status
Java Development Kit	JDK 21	Compile and run the application	To be verified
Build tool	Apache Maven	Build, dependency management, WAR packaging	To be verified
Web container	Apache Tomcat 10.1	Run the Jakarta Servlet application	To be verified
Database server	MySQL (8.0 or compatible)	Relational data storage	To be verified
JDBC driver	MySQL JDBC driver (version per build file)	Connect Java code to MySQL	To be verified
Presentation	JSP	Generate HTML pages	To be verified
Servlet API	Jakarta Servlet 6.0	Request processing	To be verified
Data access	JDBC	Execute parameterized SQL	To be verified
Version control	Git	Local source control	To be verified
Repository hosting	GitHub	Collaboration and integration	To be verified
Browser	Compatible modern web browser	Access the application	To be verified
UI framework	Bootstrap 5 (only if configured)	Responsive layout	Not confirmed

3.3 External Interface Requirements

3.3.1 User Interface

The interface shall comprise the pages listed below, with a consistent navigation bar and page structure.

Table 13. User-interface pages

Page	Purpose and principal elements
Registration page	Name, email, password, and confirmation fields with inline validation messages.
Login page	Email and password fields; a generic message on failure.

Page	Purpose and principal elements
Passenger dashboard	Entry points to search, bookings, tickets, and profile.
Flight-search page	Origin, destination, and date inputs.
Flight-results page	Flight number, route, times, fare, availability, and a select action.
Passenger-details form	Fields required by booking rules; clear marking of required fields.
Booking confirmation page	PNR, flight details, passenger details, fare, and status.
Ticket page	Ticket number, PNR, passenger and flight details, and status.
Booking-history page	The logged-in passenger's bookings with status and a cancel action where eligible.
Cancellation interface	Booking summary, explicit confirmation step, and result message.
Administrator dashboard	Summary counts and navigation to management pages; ADMIN role only.
Flight-management interface	Lists and forms for adding, editing, and deactivating flights, routes, schedules, and fares.

All pages shall use consistent terminology, provide clear success and error feedback, mark required fields, associate labels with form controls, support keyboard navigation, and adapt to supported screen sizes.

3.3.2 Software Interfaces

A browser request reaches a JSP page or a Jakarta Servlet. The servlet validates the request and calls service classes, which call DAO classes where a data-access layer is used. DAO classes obtain a JDBC connection, execute parameterized SQL against MySQL, and return Java objects. The servlet places results in the request or session and forwards to a JSP for rendering, or redirects after a state-changing request. If the implemented code uses another architecture, that architecture shall be documented in its place.

3.3.3 Communication Interfaces

The browser communicates with the server over HTTP or HTTPS. The server communicates with MySQL through JDBC using configured connection settings. HTTPS is recommended for any deployment that handles credentials or personal information; it is not claimed to be configured in the local development setup unless verified.

3.3.4 External Services

Payment gateways, Google Sign-In, email services, airline inventory APIs, and other external integrations are optional. They shall be documented as part of the system only when their integration is confirmed in the actual project.

4. Functional Requirements

Each requirement has a unique identifier and uses "shall" for mandatory behaviour. For every requirement, the tables state the primary actor, the preconditions, the main processing and expected outcome, alternative or failure conditions, the priority, and a testable acceptance condition. Priority is High (essential to the core workflow), Medium (important supporting functionality), or Optional (depends on agreed scope).

4.1 Module 1 — Authentication and User Management

Owner: Mohammed Sahil (Team Leader).

Table 14. Module 1 functional requirements

ID / Name	Description, actor, processing and outcome	Preconditions and failure conditions	Priority	Acceptance condition
FR-01 Passenger Registration	Actor: new passenger. The system shall allow a new passenger to create an account by providing full name, email, and password. The server validates required fields, email format, and password rules (minimum length and character mix to be agreed), checks that the email is not already registered, stores the salted hash and role PASSENGER, and shows a success message.	Pre: user is not logged in. Fail: missing or malformed fields, weak password, or duplicate email produce a specific validation message and no record is created.	High	Registering with valid data creates one Users row with role PASSENGER; a duplicate email is rejected.
FR-02 Passenger Login	Actor: registered user. The system shall authenticate a user by email and password. It validates input, retrieves the user record, verifies the password against the stored hash, creates a new session, and redirects to the page appropriate to the role.	Pre: account exists. Fail: empty fields, unknown email, wrong password, or (if supported) a disabled account return the same generic message "Invalid email or password" and create no session.	High	Valid credentials open a session; any invalid attempt is rejected with a non-revealing message.
FR-03 Logout	Actor: authenticated user. The system shall invalidate the current session and redirect to the login or home page, so that protected pages can no longer be reached with it.	Pre: active session. Fail: a request with an invalidated session is redirected to login.	High	After logout, requesting a protected URL redirects to login.

ID / Name	Description, actor, processing and outcome	Preconditions and failure conditions	Priority	Acceptance condition
FR-04 Password Security	Actor: system. The system shall store passwords only as output of an appropriate password-hashing algorithm (for example bcrypt, PBKDF2, or Argon2) with a unique salt per password. Plain-text storage and unsalted general-purpose hashes (such as unsalted SHA-256) are not acceptable.	Pre: registration or password change. Fail: hashing error aborts the operation and no partial record is stored.	High	Inspection of the Users table shows no plain-text passwords and distinct salts for identical passwords.
FR-05 Role-Based Access Control	Actor: system. The system shall distinguish PASSENGER and ADMIN roles. A server-side check (for example a servlet filter) shall verify the role on every protected request, regardless of whether the link is displayed.	Pre: authenticated session. Fail: a PASSENGER requesting an ADMIN resource receives an access-denied response and no data is returned.	High	A passenger who enters an admin URL directly is denied.
FR-06 Passenger Profile	Actor: authenticated passenger. The system shall let the passenger view their profile and update permitted fields (for example name and contact details). Email and role changes are not permitted through this function.	Pre: logged in. Fail: invalid values are rejected with field-level messages; unchanged data is preserved.	Medium	A valid update is saved and visible on reload; an invalid update changes nothing.
FR-07 Session Management	Actor: system. The system shall maintain authenticated sessions, create a new session identifier at login, expire idle sessions after a configured period, and deny protected resources to unauthenticated requests.	Pre: user has logged in. Fail: expired or missing session redirects to login with a message.	High	An idle session beyond the timeout is rejected; protected pages require a valid session.
FR-08 Input Validation	Actor: system. The system shall validate all user input on the server (presence, type, length, format, range) and return field-level feedback. Client-side checks may assist but are not relied upon.	Pre: any form submission. Fail: invalid input is not processed and the form is redisplayed with messages.	High	Submissions that bypass client checks are still rejected by the server.

4.2 Module 2 — Flight Search and Availability

Owner: Nikhil Ranjan Choubey.

Table 15. Module 2 functional requirements

ID / Name	Description, actor, processing and outcome	Preconditions and failure conditions	Priority	Acceptance condition
FR-09 Search Flights	Actor: passenger. The system shall allow search by origin, destination, and travel date. It validates that all three are present, that the date is a valid date not in the past, and that origin and destination differ.	Pre: search page is open. Fail: missing input, invalid or past date, or identical origin and destination return a validation message and no query is run.	High	A valid search returns flights matching all three criteria; invalid input shows a message.
FR-10 Display Flight Results	Actor: passenger. The system shall display, for each matching flight, the flight number, origin, destination, departure and arrival times, fare, and availability, where those fields exist in the data.	Pre: valid search. Fail: a database error shows a controlled error page.	High	Displayed values equal the stored values for each matching flight.
FR-11 Filter and Sort Results	Actor: passenger. Optional: the system may allow sorting by fare, departure time, or arrival time, and filtering by implemented attributes. Any such function shall be marked as implemented only when verified.	Pre: results displayed. Fail: an unsupported option is ignored safely.	Optional	Sorting by fare returns results in non-decreasing fare order.
FR-12 Availability Information	Actor: passenger. The system shall display the number of seats available, derived from the application's database (capacity or remaining seats minus active bookings). This is not real-time airline inventory.	Pre: flight exists. Fail: if availability cannot be computed, the field is shown as unavailable and booking is not offered.	High	Displayed availability equals the database value after a booking or cancellation.
FR-13 Invalid Search Handling	Actor: passenger. The system shall show a clear message when parameters are invalid or when no flight matches.	Pre: search submitted. Fail: none; the page stays usable.	Medium	A search with no matching data shows a "no flights found" message.
FR-14 Flight Selection	Actor: passenger. The system shall let the passenger select a flight and proceed to booking. Only the flight identifier is passed on; the server re-validates that the flight exists, is ACTIVE, and has availability.	Pre: results displayed; user logged in (otherwise redirected to login). Fail: a stale, inactive, or tampered identifier returns an error and no booking form.	High	A modified or inactive flight identifier is rejected.

4.3 Module 3 — Booking and Passenger Management

Owner: Mukesh Sah.

Table 16. Module 3 functional requirements

ID / Name	Description, actor, processing and outcome	Preconditions and failure conditions	Priority	Acceptance condition
FR-15 Passenger Details	Actor: passenger. The system shall capture the passenger information required by business rules (for example name and age; gender and contact details only if required) and validate each field. Only necessary personal data shall be collected.	Pre: valid flight selected. Fail: invalid or missing fields redisplay the form with messages.	High	Invalid details do not create a booking; valid details are accepted.
FR-16 Booking Creation	Actor: authenticated passenger. The system shall create a booking linked to the authenticated user and the selected flight, with initial status CONFIRMED (no payment step is in baseline scope).	Pre: logged in; flight valid. Fail: any failed step prevents creation (see FR-23).	High	A successful booking row references the correct user and flight.
FR-17 PNR Generation	Actor: system. The system shall generate a unique booking reference for each successful booking, using a random alphanumeric code, and rely on a UNIQUE database constraint; on collision it shall regenerate and retry.	Pre: booking validated. Fail: repeated collision aborts with a controlled error.	High	No two bookings share a PNR; the UNIQUE constraint rejects duplicates.
FR-18 Seat Allocation	Actor: system. Where seat-level allocation is implemented, the system shall allocate a seat according to configured rules and prevent two active bookings from holding the same seat on the same flight. A graphical seat map is not assumed. Where only seat counts are kept, the system shall decrement availability atomically.	Pre: availability confirmed inside the transaction. Fail: a conflicting seat or no remaining seats rejects the booking.	High	Concurrent requests for the last seat result in one booking only.
FR-19 Booking Confirmation	Actor: passenger. The system shall display the PNR, flight details, passenger details, fare, and booking status after a successful booking.	Pre: booking committed. Fail: if display fails after commit, the booking remains retrievable by PNR or history.	High	Confirmation values equal the stored booking data.
FR-20 Booking Persistence	Actor: system. The system shall save booking, passenger, and availability changes in one database transaction, committing only when all succeed.	Pre: validation passed. Fail: any exception triggers rollback.	High	After a forced failure, no partial booking or passenger rows exist.

ID / Name	Description, actor, processing and outcome	Preconditions and failure conditions	Priority	Acceptance condition
FR-21 Duplicate Booking Protection	Actor: system. Where supported, the system shall prevent repeated form submissions (for example double-click or refresh) from creating duplicate bookings, using a one-time form token or redirect-after-post.	Pre: booking form submitted. Fail: a repeated token is rejected with a message pointing to the existing booking.	Medium	Submitting the same form twice yields one booking.
FR-22 Booking Validation	Actor: system. Before commit the system shall verify that the flight exists, is ACTIVE, has not departed, and has sufficient availability, and that the user is authenticated.	Pre: booking request received. Fail: any failed check aborts with a specific message.	High	Booking an inactive, past, or full flight is rejected.
FR-23 Booking Failure Handling	Actor: system. The system shall handle failed transactions, unavailable flights, invalid passenger data, and database errors by rolling back, showing a safe message, and logging details without exposing internals.	Pre: a failure occurs. Fail: not applicable.	High	Error pages contain no stack trace, SQL, or credentials.

4.4 Module 4 — Ticketing and Cancellation

Owner: Muskan Raj.

Table 17. Module 4 functional requirements

ID / Name	Description, actor, processing and outcome	Preconditions and failure conditions	Priority	Acceptance condition
FR-24 Ticket Generation	Actor: passenger. The system shall generate or display a ticket for a confirmed booking showing the PNR, ticket number, passenger names, flight details, fare, and status, as an on-screen page. PDF output is not claimed unless implemented.	Pre: booking CONFIRMED. Fail: a booking that is not confirmed shows no ticket.	High	A ticket page shows the data of the correct booking.
FR-25 Booking History	Actor: authenticated passenger. The system shall list only the logged-in passenger's bookings, with status, newest first.	Pre: logged in. Fail: no bookings shows an empty-state message.	Medium	History contains only the user's own bookings.

ID / Name	Description, actor, processing and outcome	Preconditions and failure conditions	Priority	Acceptance condition
FR-26 PNR-Based Lookup	Actor: permitted user. The system shall retrieve a booking by PNR, subject to authorization: a passenger may retrieve only their own booking; an administrator may retrieve any booking.	Pre: logged in. Fail: an unknown PNR gives "not found"; another passenger's PNR gives the same response without revealing existence.	High	A valid own PNR returns the booking; another user's PNR does not.
FR-27 Ticket Details	Actor: permitted user. The system shall display the details associated with a valid booking: passenger records, flight, fare, booking and ticket status.	Pre: booking retrieved and authorized. Fail: missing related record shows a controlled error.	High	All associated data match the database.
FR-28 Cancellation Request	Actor: passenger. The system shall let an eligible passenger request cancellation of their own booking, showing a confirmation step before processing.	Pre: booking CONFIRMED and owned by the user. Fail: ineligible bookings show the reason.	High	An eligible booking can be cancelled after confirmation.
FR-29 Cancellation Validation	Actor: system. The system shall verify ownership, current status, and eligibility (for example the flight has not departed; the specific rule is to be defined) before cancelling.	Pre: request received. Fail: not owner, already cancelled, or ineligible are rejected.	High	Each failed check blocks cancellation.
FR-30 Booking Status Update	Actor: system. On successful cancellation the system shall set the booking status and ticket status to CANCELLED in one transaction and restore availability.	Pre: validation passed. Fail: any update failure rolls back all changes.	High	Statuses and availability change together or not at all.
FR-31 Duplicate Cancellation Prevention	Actor: system. The system shall prevent repeated cancellation from changing data again, by conditioning the update on the current status.	Pre: booking already CANCELLED. Fail: the system returns "already cancelled".	High	A second cancellation changes no rows and does not restore availability twice.
FR-32 Cancellation Feedback	Actor: passenger. The system shall display a clear success or failure message reporting the outcome.	Pre: cancellation attempted. Fail: not applicable.	Medium	The message matches the actual database outcome.

ID / Name	Description, actor, processing and outcome	Preconditions and failure conditions	Priority	Acceptance condition
FR-33 Administrative Booking Review	Actor: administrator. The system shall let an authorized administrator review booking and cancellation records. Refunds are not part of the baseline and are not described unless later specified.	Pre: ADMIN session. Fail: non-admin access is denied.	High	An admin sees all bookings; a passenger cannot.

4.5 Module 5 — Admin Dashboard and Flight Management

Owner: Prashant Chaudhary.

Table 18. Module 5 functional requirements

ID / Name	Description, actor, processing and outcome	Preconditions and failure conditions	Priority	Acceptance condition
FR-34 Administrator Dashboard	Actor: administrator. The system shall show summary figures that can be computed accurately from stored data, such as number of flights, bookings, cancellations, and active flights.	Pre: ADMIN session. Fail: non-admin access is denied.	High	Each figure equals a count taken directly from the database.
FR-35 Add Flight	Actor: administrator. The system shall let an administrator create a flight record with flight number, origin, destination, departure and arrival times, fare, and seat capacity, after validation.	Pre: ADMIN session. Fail: invalid data, arrival before departure, non-positive fare or capacity, or a duplicate flight/time is rejected.	High	A valid flight appears in search results; an invalid one is not saved.
FR-36 Update Flight	Actor: administrator. The system shall let an administrator modify permitted attributes. Changes that would invalidate existing bookings (for example reducing capacity below booked seats) shall be rejected.	Pre: flight exists. Fail: validation or integrity failure leaves the record unchanged.	High	Updated values appear in search; unsafe changes are blocked.
FR-37 Deactivate or Remove Flight	Actor: administrator. The system shall deactivate (set status INACTIVE) rather than delete any flight referenced by bookings, so that history is preserved. Hard deletion is permitted only for flights with no bookings.	Pre: flight exists. Fail: attempted deletion of a referenced flight is converted to deactivation or rejected.	High	A deactivated flight is not offered for booking but remains in history.

ID / Name	Description, actor, processing and outcome	Preconditions and failure conditions	Priority	Acceptance condition
FR-38 Route Management	Actor: administrator. Where route data is stored, the system shall let an administrator add, edit, and deactivate routes (origin–destination pairs).	Pre: ADMIN session. Fail: duplicate or invalid routes are rejected.	High	A new route can be used when adding a flight.
FR-39 Schedule Management	Actor: administrator. The system shall let an administrator manage departure and arrival schedules, validating date-time formats and ordering.	Pre: flight exists. Fail: invalid date-times are rejected.	High	A schedule change is reflected in search results.
FR-40 Fare Management	Actor: administrator. The system shall let an administrator create or update fares, requiring a positive decimal value within an agreed range.	Pre: flight exists. Fail: zero, negative, or non-numeric fares are rejected.	Medium	An updated fare is shown for new bookings; existing bookings keep their stored fare.
FR-41 Booking Management	Actor: administrator. The system shall let an administrator view booking records within their permissions, without permitting unauthorized modification.	Pre: ADMIN session. Fail: non-admin access is denied.	High	An admin can list bookings; a passenger cannot.
FR-42 Reports	Actor: administrator. The system shall produce administrative reports derived from real database records (for example bookings per flight or cancellation counts). Reports not implemented are future work.	Pre: ADMIN session. Fail: empty data shows an empty report.	Medium	Report totals agree with direct database queries.

5. Non-Functional Requirements

Each non-functional requirement is identified (NFR-xx) and phrased so that it can be verified by inspection, test, or measurement.

5.1 Security

- **NFR-01 Password hashing.** Passwords shall be stored using a salted, adaptive hashing algorithm (see FR-04).
- **NFR-02 Authorization.** Every protected request shall be authorized on the server using the role in the session (FR-05).
- **NFR-03 Session security.** A new session identifier shall be issued at login; sessions shall expire after inactivity and be invalidated at logout; cookies shall be marked HttpOnly (and Secure under HTTPS).

- **NFR-04 Server-side validation.** All input shall be validated on the server before use (FR-08).
- **NFR-05 SQL injection.** All SQL shall use parameterized (prepared) statements; string concatenation of user input into SQL is not permitted.
- **NFR-06 Cross-site scripting.** All dynamic output shall be encoded for its context (HTML, attribute, JavaScript, URL).
- **NFR-07 CSRF.** State-changing requests (booking, cancellation, profile and admin changes) shall be protected by an anti-CSRF token or an equivalent mechanism, where applicable.
- **NFR-08 Error messages.** Error responses shall not reveal stack traces, SQL, file paths, or credentials.
- **NFR-09 Personal data.** Personal data shall be accessible only to its owner and authorized administrators.
- **NFR-10 Administrative access.** Administrative operations shall be reachable only by ADMIN users.
- **NFR-11 Configuration secrets.** Database credentials and other secrets shall be kept in configuration outside public source control.

5.2 Performance

The following are proposed targets for a local academic deployment with a small, defined data set. They are targets to be measured, not verified results.

- **NFR-12.** Typical page requests should complete within about 3 seconds under a defined test load (to be specified, for example 10 concurrent users).
- **NFR-13.** Flight searches should return within about 2 seconds on a data set of a stated size (to be specified).
- **NFR-14.** Common operations should not issue unnecessary queries; search and listing queries should select only needed columns and use indexes on search fields.

5.3 Reliability and Data Integrity

- **NFR-15.** PNR values shall be unique, enforced by a database constraint.
- **NFR-16.** Referential integrity shall be enforced through foreign keys.
- **NFR-17.** Booking creation and cancellation shall be transactional.
- **NFR-18.** Booking and ticket status shall change only through defined transitions (CONFIRMED to CANCELLED; ISSUED to CANCELLED).
- **NFR-19.** Where seat-level allocation is used, conflicting allocations shall be prevented by a uniqueness rule combined with a transactional availability check.
- **NFR-20.** Historical booking records shall be preserved; flights referenced by bookings shall be deactivated, not deleted.

- **NFR-21.** Failures shall result in controlled error messages and no corrupted data.

5.4 Usability

NFR-22. Navigation shall be consistent across pages. **NFR-23.** Forms shall mark required fields and show understandable validation messages next to the field. **NFR-24.** Important actions (booking, cancellation) shall give explicit confirmation and result feedback. **NFR-25.** A new passenger should be able to register, search, and book without external instructions (to be confirmed by usability testing).

5.5 Maintainability

NFR-26. Each module shall have a clear responsibility and documented interface. **NFR-27.** Presentation, request handling, business logic, and data access shall be separated. **NFR-28.** Naming conventions and package structure shall be consistent across modules. **NFR-29.** Source code shall be under Git version control, with changes integrated through reviewed branches. **NFR-30.** Database access shall follow one consistent pattern (for example DAO with prepared statements).

5.6 Compatibility

NFR-31. The application shall build with Java 21 and Maven, run on Apache Tomcat 10.1 using the Jakarta Servlet API 6.0, and use MySQL 8.0 or a compatible verified version. **NFR-32.** It shall function in the current versions of at least two mainstream browsers, to be named and tested; universal compatibility is not claimed.

5.7 Scalability

A modular structure and a relational database with indexed search fields allow data volume and features to grow, for example by adding payment or notification modules. Scalability to production traffic is not claimed and would require separate load testing, connection pooling, and deployment design.

5.8 Privacy

NFR-33. Only necessary passenger information shall be collected. **NFR-34.** Booking and profile data shall be accessible only according to user permissions. **NFR-35.** Logs shall not contain passwords or session tokens. **NFR-36.** Demonstrations and screenshots shall use test data.

5.9 Availability and Recovery

NFR-37. If the database is unavailable the application shall show a controlled error page and shall not corrupt data. **NFR-38.** The team shall document a manual backup and restore procedure for the MySQL database (for example a scheduled export). Automated backups are not claimed unless implemented and verified.

5.10 Accessibility

NFR-39. Forms shall be operable by keyboard, with labels bound to controls. **NFR-40.** Text and controls shall have sufficient colour contrast. **NFR-41.** Validation feedback shall be understandable and not conveyed by colour alone.

6. System Architecture and UML Diagrams

Each figure below is a proposed design derived from this specification. The application source code has not been inspected for this document; diagrams must be reconciled with the implemented classes, pages, and database before final submission. The Mermaid source of every diagram is given in the Markdown version of this document. Class names used in the diagrams are conceptual and are not claimed to exist in the repository.

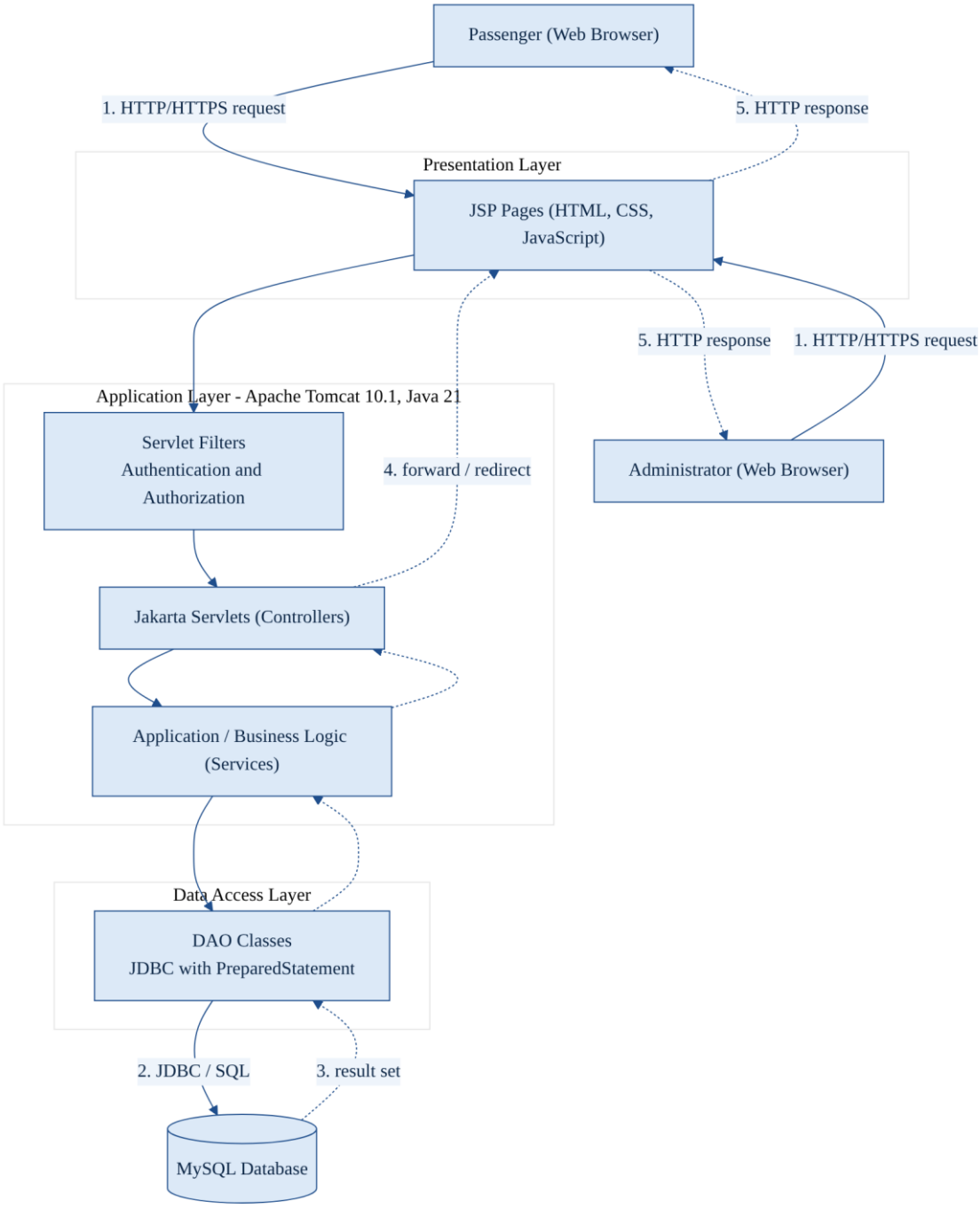


Figure 1. High-Level System Architecture (proposed)

Purpose: to show the layers and the flow of a request and its response. Passenger and Administrator browsers send HTTP or HTTPS requests to JSP pages; servlet filters check authentication and role; servlets call business-logic services; DAO classes use JDBC to query MySQL; results return along the same path and the servlet forwards or redirects to a JSP that renders the response.

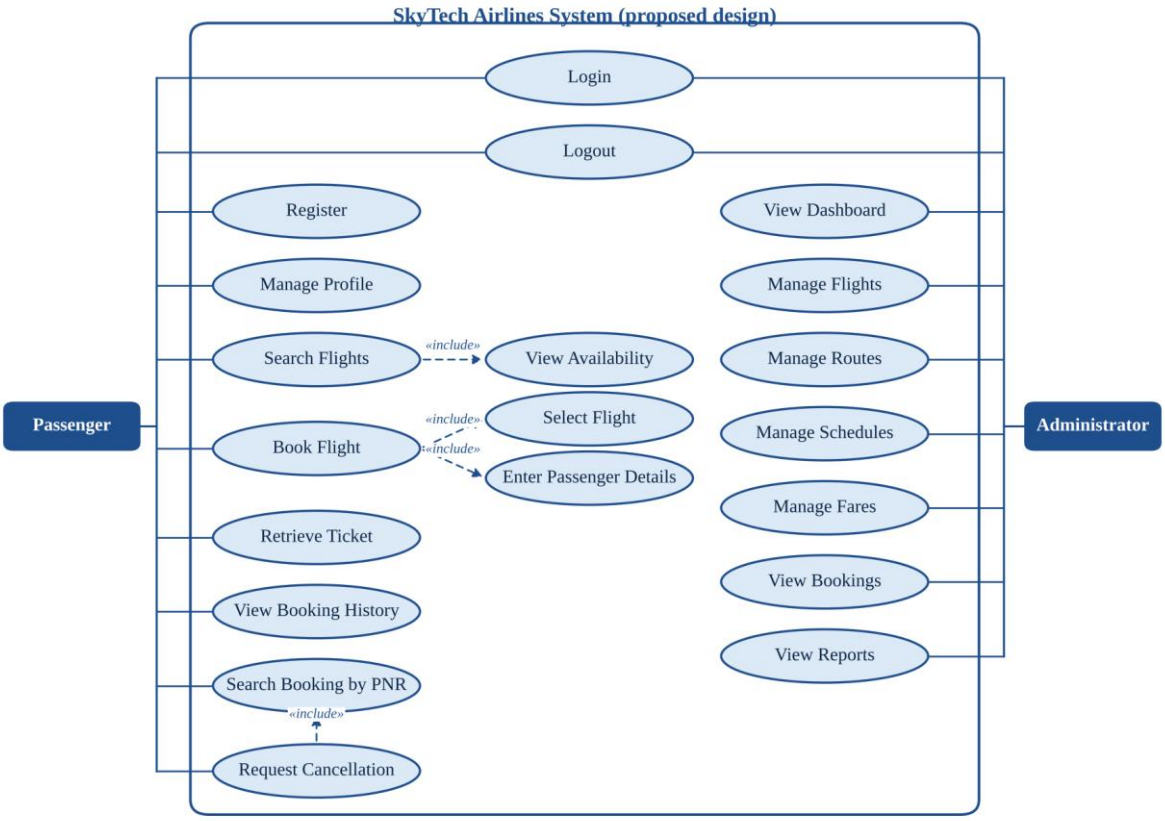


Figure 2. UML Use-Case Diagram (proposed)

Purpose: to show the two actors and their interactions with the system. Login and Logout are shared by both actors. Search Flights includes View Availability; Book Flight includes Select Flight and Enter Passenger Details; Request Cancellation includes Search Booking by PNR. Include relationships are shown only where the second use case is always performed as part of the first. Every use case is subject to authentication and authorization.

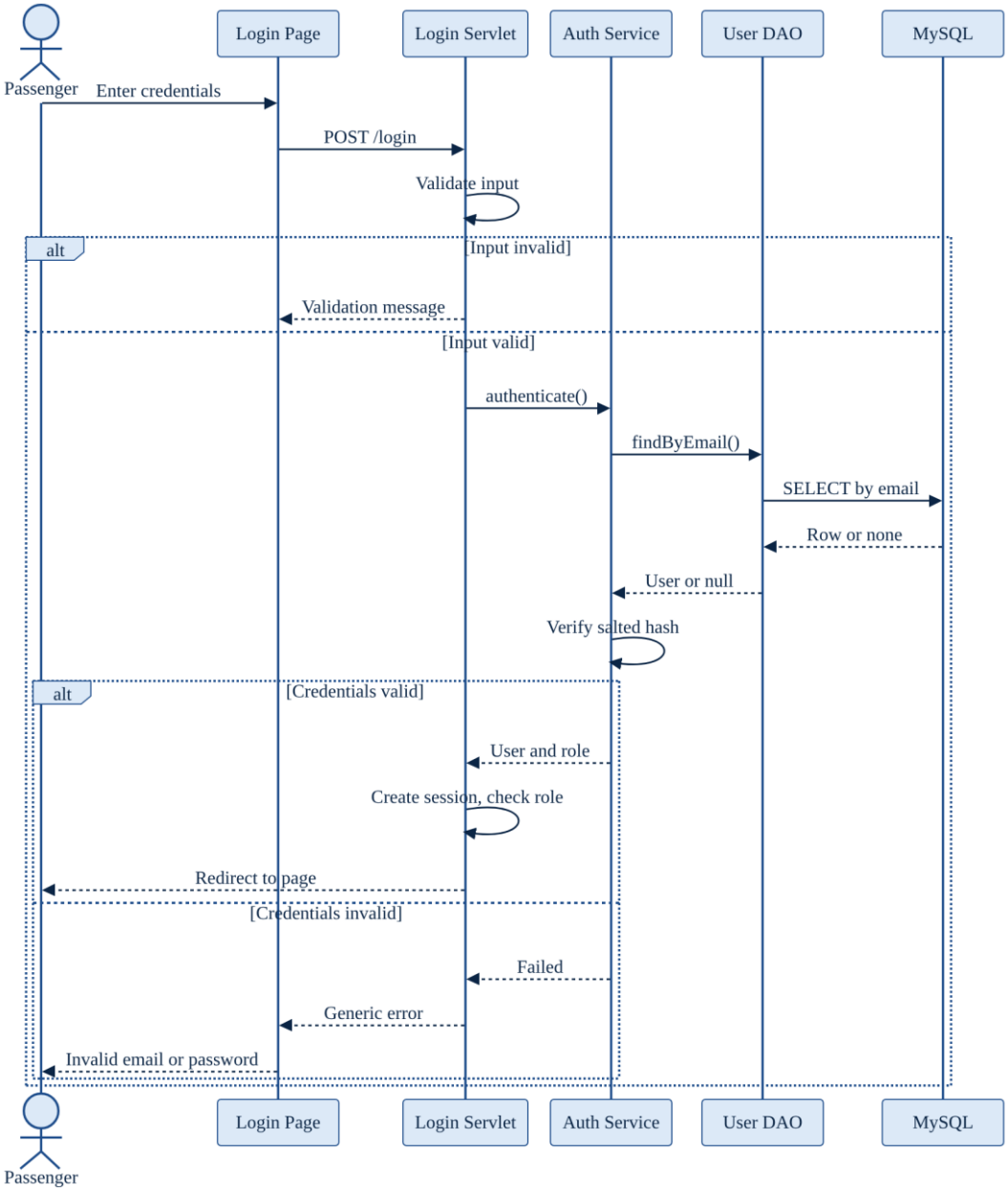


Figure 3. Passenger Login Sequence Diagram (proposed)

Purpose: to show how credentials are processed. Invalid input is rejected immediately. For valid input the authentication service loads the user record through the DAO, verifies the password against the stored salted hash, and on success the servlet creates a new session and redirects according to role. On failure a generic message is returned so that the response does not reveal whether the email exists.

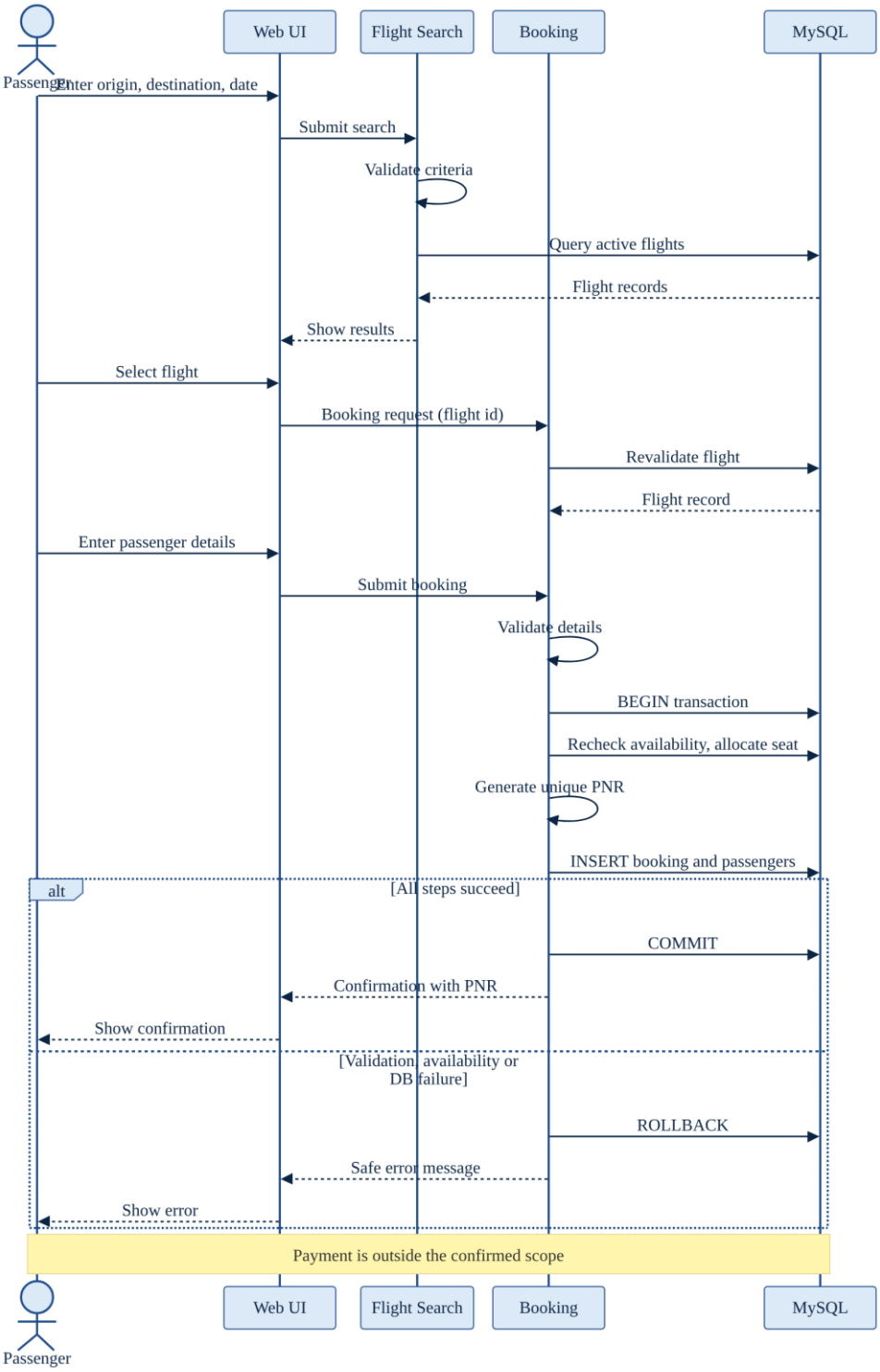


Figure 4. Flight Search and Booking Sequence Diagram (proposed)

Purpose: to show the booking flow and its transactional behaviour. Search criteria are validated and matching flights displayed. After selection the flight is revalidated on the server, passenger details are validated, and a transaction rechecks availability, allocates a seat, generates a unique PNR, and inserts the booking and passenger records. Success commits and returns a confirmation; any failure rolls back. Payment processing is outside the confirmed scope.

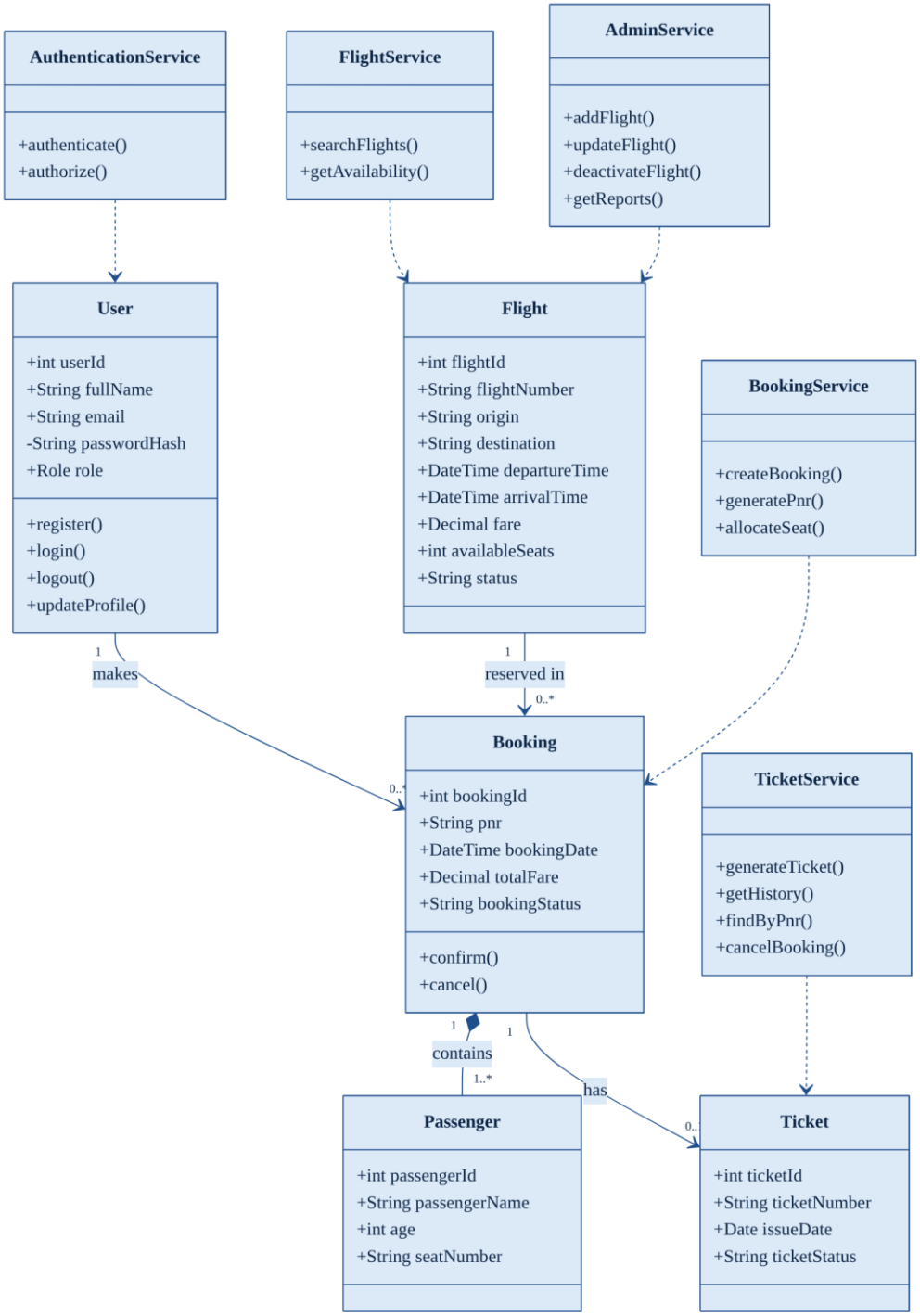


Figure 5. Conceptual Class Diagram (proposed)

Purpose: to show the principal business entities and services and their relationships. A User makes zero or more Bookings; a Flight is reserved in zero or more Bookings; a Booking contains one or more Passengers and has at most one Ticket. Service classes group operations by module. These are conceptual classes, not a statement about the Java source.

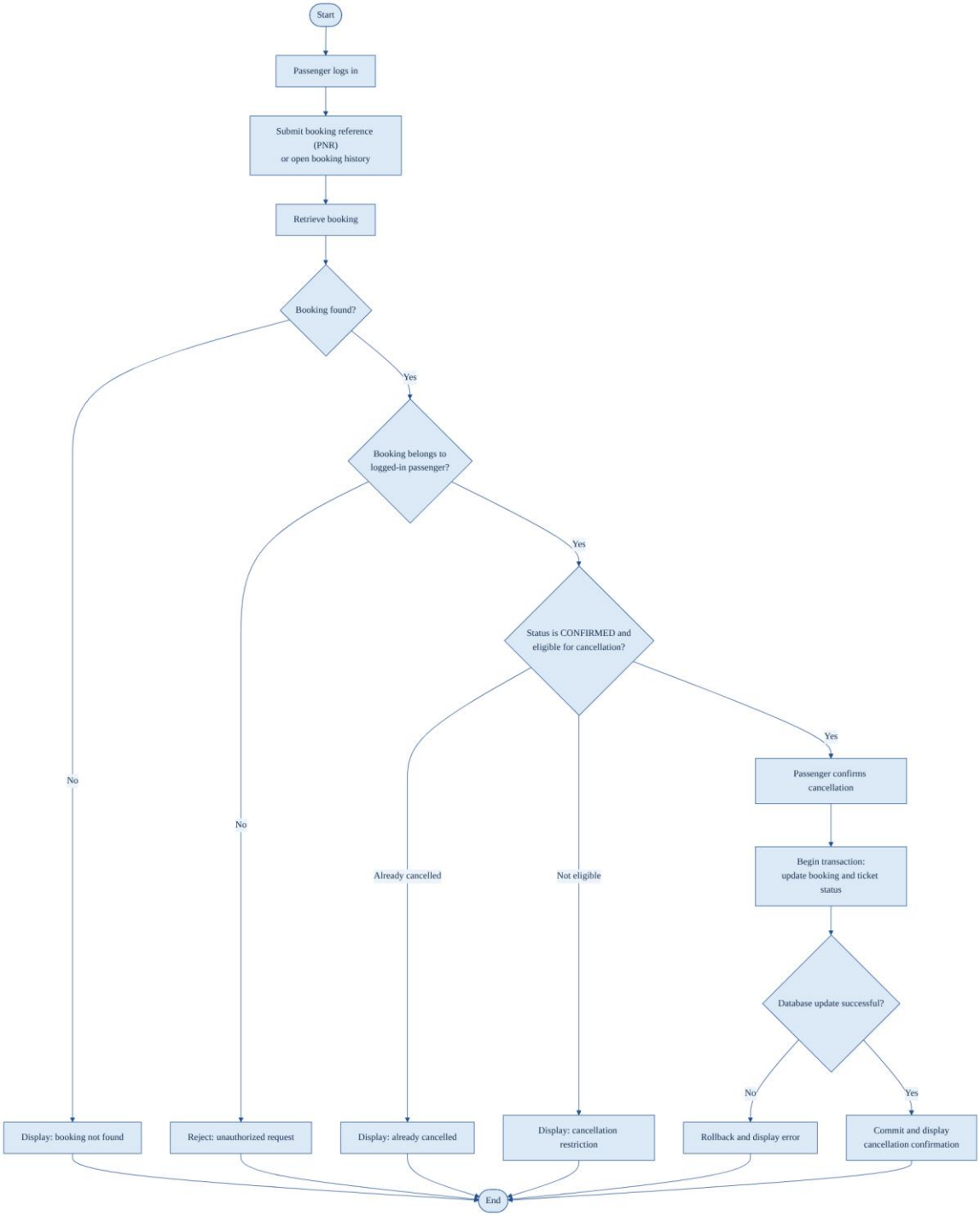


Figure 6. Ticket Cancellation Activity Diagram (proposed)

Purpose: to show the decision logic of cancellation. After authentication and PNR submission the booking is retrieved; unknown references, bookings owned by another user, bookings already cancelled, and ineligible bookings each end with a specific message. An eligible booking is cancelled after the passenger confirms, inside a transaction that updates booking and ticket status; a failed update is rolled back. Refunds are not modelled.

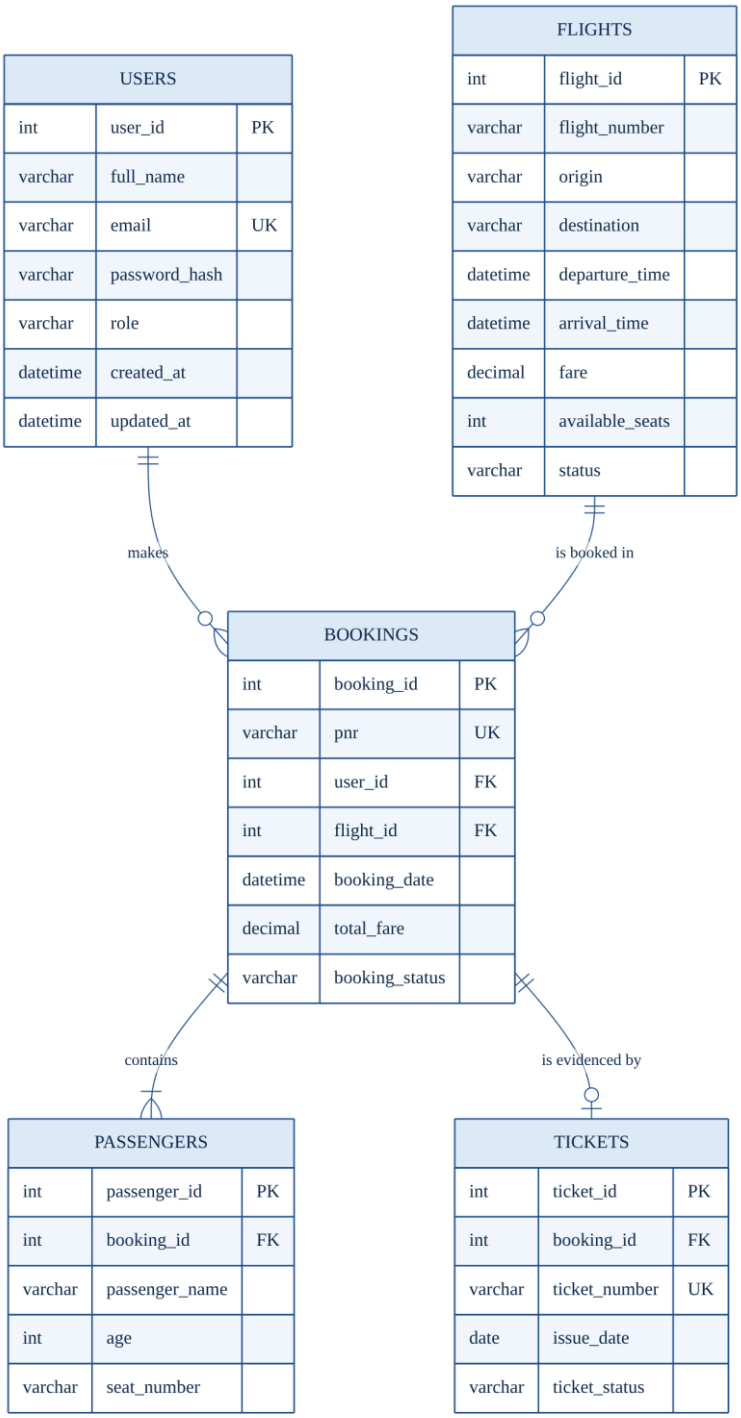


Figure 7. Entity-Relationship Diagram (proposed conceptual)

Purpose: to show the proposed entities with primary keys (PK), foreign keys (FK), and unique keys (UK). One user has many bookings; one flight has many bookings; a booking has one or more passengers and at most one ticket. A composite unique constraint on flight number and departure time is proposed for Flights. This diagram has not been checked against the actual SQL schema.

7. Database Requirements and Data Dictionary

The database maintains users, flights, bookings, passengers, and tickets and is the single source of truth for availability and status. The structure below is a proposed conceptual design, to be reconciled with the actual schema file (schema.sql) before it is treated as the implemented design. Tables, fields, and status values are used identically in every section of this document.

7.1 Users Table

Fields: user_id (primary key), full_name, email, password_hash, role, created_at, updated_at. Email shall be unique and stored in a normalized form; role shall be restricted to PASSENGER or ADMIN; password_hash holds only the salted hash produced under FR-04.

7.2 Flights Table

Fields: flight_id (primary key), flight_number, origin, destination, departure_time, arrival_time, fare, available_seats, status. Origin and destination shall differ; arrival_time shall be later than departure_time; fare shall be positive; available_seats shall not be negative; status shall be ACTIVE or INACTIVE; flight_number combined with departure_time shall be unique. A total-capacity field may be added if seat counts are derived rather than stored.

7.3 Bookings Table

Fields: booking_id (primary key), pnr (unique), user_id (foreign key to Users), flight_id (foreign key to Flights), booking_date, total_fare, booking_status. booking_status shall be CONFIRMED or CANCELLED. A booking belongs to exactly one user and one flight, and total_fare is stored at booking time so that later fare changes do not alter history.

7.4 Passengers Table

Fields: passenger_id (primary key), booking_id (foreign key to Bookings), passenger_name, age, seat_number (nullable). Gender and contact details shall be added only if business rules require them, to avoid collecting unnecessary personal data. A booking has one or more passengers.

7.5 Tickets Table

Fields: ticket_id (primary key), booking_id (foreign key to Bookings, unique), ticket_number (unique), issue_date, ticket_status. ticket_status shall be ISSUED or CANCELLED. One ticket exists per confirmed booking and remains linked to it after cancellation so that history is retained.

7.6 Data Dictionary

The types and names below are suggestions and shall be reconciled with the actual schema file before finalization.

Table 19. Proposed data dictionary

Table	Field	Suggested type	Key	Null?	Constraints	Description
Users	user_id	INT AUTO_INCREMENT	PK	No	Primary key	Unique user identifier
Users	full_name	VARCHAR(100)	—	No	Non-empty	Passenger or administrator name
Users	email	VARCHAR(255)	—	No	UNIQUE	Login identifier
Users	password_hash	VARCHAR(255)	—	No	Salted hash only	Stored password hash
Users	role	VARCHAR(20)	—	No	CHECK IN ('PASSENGER','ADMIN')	Authorization role
Users	created_at	DATETIME	—	No	Default current time	Account creation time
Users	updated_at	DATETIME	—	No	Updated on change	Last modification time
Flights	flight_id	INT AUTO_INCREMENT	PK	No	Primary key	Unique flight identifier
Flights	flight_number	VARCHAR(10)	—	No	UNIQUE with departure_time	Airline flight number
Flights	origin	VARCHAR(60)	—	No	≠ destination	Departure city or airport
Flights	destination	VARCHAR(60)	—	No	≠ origin	Arrival city or airport
Flights	departure_time	DATETIME	—	No	Valid date-time	Scheduled departure
Flights	arrival_time	DATETIME	—	No	> departure_time	Scheduled arrival
Flights	fare	DECIMAL(10,2)	—	No	CHECK > 0	Fare per passenger
Flights	available_seats	INT	—	No	CHECK ≥ 0	Seats remaining
Flights	status	VARCHAR(20)	—	No	CHECK IN ('ACTIVE','INACTIVE')	Bookable state
Bookings	booking_id	INT AUTO_INCREMENT	PK	No	Primary key	Unique booking identifier
Bookings	pnr	VARCHAR(20)	—	No	UNIQUE	Booking reference

Table	Field	Suggested type	Key	Null?	Constraints	Description
Bookings	user_id	INT	FK → Users	No	Foreign key	Booking owner
Bookings	flight_id	INT	FK → Flights	No	Foreign key	Booked flight
Bookings	booking_date	DATETIME	—	No	Default current time	Booking time
Bookings	total_fare	DECIMAL(10,2)	—	No	CHECK ≥ 0	Fare at booking time
Bookings	booking_status	VARCHAR(20)	—	No	CHECK IN ('CONFIRMED','CANCELLED')	Reservation state
Passengers	passenger_id	INT AUTO_INCREMENT	PK	No	Primary key	Unique passenger record
Passengers	booking_id	INT	FK → Bookings	No	Foreign key	Owning booking
Passengers	passenger_name	VARCHAR(100)	—	No	Non-empty	Traveller name
Passengers	age	INT	—	No	CHECK 0–120	Traveller age
Passengers	seat_number	VARCHAR(5)	—	Yes	Unique per flight when active	Allocated seat
Tickets	ticket_id	INT AUTO_INCREMENT	PK	No	Primary key	Unique ticket identifier
Tickets	booking_id	INT	FK → Bookings	No	UNIQUE, foreign key	Related booking
Tickets	ticket_number	VARCHAR(20)	—	No	UNIQUE	Printed ticket number
Tickets	issue_date	DATE	—	No	Default current date	Issue date
Tickets	ticket_status	VARCHAR(20)	—	No	CHECK IN ('ISSUED','CANCELLED')	Ticket state

7.7 Database Integrity Rules

- Primary-key values shall uniquely identify rows in every table.
- Foreign keys shall prevent a booking without a valid user and flight, a passenger without a booking, and a ticket without a booking.
- The pnr column shall be UNIQUE; the application shall regenerate on collision.
- Required fields shall be NOT NULL and non-empty after trimming.
- Fare shall be positive and available_seats shall not be negative.
- booking_status, ticket_status, flight status, and role shall be restricted to the values defined in this section.
- Ticket status shall agree with booking status: a CANCELLED booking shall have a CANCELLED ticket.
- Booking creation and cancellation shall be executed in single transactions.

- Rows referenced by historical bookings shall not be hard-deleted; flights shall be deactivated instead.
- Seat uniqueness, where seat-level allocation exists, shall be enforced for active bookings on the same flight, through a unique constraint on flight and seat (which may require a flight reference in the seat record) together with the transactional availability check.

8. Team Responsibilities and Module Integration

Table 20. Module responsibilities

Module	Name	Owner	Key responsibilities
1	Authentication & User Management	Mohammed Sahil, Team Leader	Registration, login/logout, profiles, roles, sessions, authorization
2	Flight Search & Availability	Nikhil Ranjan Choubey	Search criteria, flight results, schedule display, fare and availability display
3	Booking & Passenger Management	Mukesh Sah	Passenger details, booking creation, PNR generation, seat allocation, confirmation
4	Ticketing & Cancellation	Muskan Raj	Ticket display, booking history, PNR lookup, cancellation, status updates
5	Admin Dashboard & Flight Management	Prashant Chaudhary	Dashboard, flight CRUD, route/schedule/fare management, booking review, reports

Each module shall expose clearly defined interfaces (servlet URLs, service methods, and shared session attributes) and shall use the same database identifiers, status values, and business rules defined in Section 7.

8.1 Integration Dependencies

- Module 1 provides authentication and authorization to every other module.
- Module 2 provides valid flight-search results and the selected flight identifier to Module 3.
- Module 3 depends on authenticated users (Module 1) and valid flights (Modules 2 and 5).
- Module 4 depends on booking and passenger records created by Module 3.
- Module 5 maintains the flight data used by Modules 2 and 3 and reviews booking information from Modules 3 and 4.

8.2 Shared Development Responsibilities

Table 21. Shared responsibilities

Area	Agreed practice (proposed)
Git branch management	One branch per module or feature; the main branch is integrated only through reviewed merges.
Code reviews	Each merge request is reviewed by at least one other member.
Naming conventions	Common package, class, table, and column naming rules, as in Section 7.
Database schema coordination	A single schema file owned by the team; changes announced before merge.
Configuration management	Connection settings and secrets kept in local configuration, not committed.
Conflict resolution	Merge conflicts resolved by the owners of the affected modules; the Team Leader decides if unresolved.
Integration testing	Performed jointly against the scenarios in Section 9.
Documentation	Each owner documents their module; the Team Leader maintains this SRS.
Communication	Regular team meetings and a shared channel; interface changes communicated to dependent owners.

This section describes the agreed way of working. It makes no statement that any member has completed any task; completion shall be supported by repository evidence.

9. Verification, Validation and Test Cases

The test strategy comprises: unit testing of validation, hashing, PNR generation, and status transitions by each module owner; integration testing of module interfaces; system testing of end-to-end passenger and administrator workflows; security testing (Section 9.1); usability testing against Section 5.4; and regression testing, repeating the critical cases after each integration. Test outcomes shall be recorded only after execution; every case below is currently unexecuted.

Table 22. Functional test cases (all unexecuted)

Test ID	Module	Test scenario	Preconditions	Test steps	Expected result	Actual result	Status
TC-01	1	Successful registration	Email not registered	1. Open registration. 2. Enter valid name, email, strong password. 3. Submit.	Account created; one Users row with role PASSENGER; stored value is a hash; success message shown.	Not executed	Not run
TC-02	1	Duplicate email registration	Email already registered	1. Open registration. 2. Enter the existing email. 3. Submit.	Registration rejected with a duplicate-email message; no new row.	Not executed	Not run
TC-03	1	Login with valid credentials	Registered passenger	1. Open login. 2. Enter valid email and password. 3. Submit.	Session created; redirect to passenger dashboard.	Not executed	Not run
TC-04	1	Login with invalid credentials	Registered passenger	1. Enter valid email with wrong password. 2. Submit. 3. Repeat with unknown email.	Both attempts show the same generic message; no session created.	Not executed	Not run
TC-05	1	Logout and protected-page access	Logged-in passenger	1. Click logout. 2. Press browser Back. 3. Request a protected URL.	Session invalidated; protected URL redirects to login.	Not executed	Not run
TC-06	1	Passenger attempts admin access	Logged-in passenger	1. Enter the admin dashboard URL directly.	Access denied; no admin data returned.	Not executed	Not run
TC-07	2	Valid flight search	Active flights exist for route and date	1. Enter origin, destination, date. 2. Submit.	All active matching flights shown with correct times, fare, and availability.	Not executed	Not run
TC-08	2	Search with missing fields	Search page open	1. Leave destination empty. 2. Submit.	Validation message; no query executed.	Not executed	Not run
TC-09	2	Search with no matching flights	No flights for chosen route/date	1. Search a route/date with no data.	"No flights found" message shown.	Not executed	Not run
TC-10	3	Booking with valid passenger details	Logged in; flight with free seats	1. Select flight. 2. Enter valid details. 3. Submit.	Booking CONFIRMED; unique PNR shown; availability reduced by passenger count.	Not executed	Not run
TC-11	3	Duplicate booking submission	Booking form submitted once	1. Submit the same form twice (double-click or refresh).	Only one booking created; second submission is rejected or redirected.	Not executed	Not run
TC-12	3	Unique PNR generation	Several bookings created	1. Create at least 20 bookings. 2. Compare PNR values.	All PNR values distinct; duplicate insert is rejected by constraint.	Not executed	Not run

Test ID	Module	Test scenario	Preconditions	Test steps	Expected result	Actual result	Status
TC-13	3	Booking persistence	TC-10 completed	1. Log out and in. 2. Open booking history.	Booking, passengers, and fare appear as stored.	Not executed	Not run
TC-14	4	Ticket retrieval with valid reference	Passenger owns confirmed booking	1. Enter own PNR in lookup.	Ticket and booking details displayed correctly.	Not executed	Not run
TC-15	4	Lookup with invalid reference	Logged in	1. Enter a non-existent PNR.	"Booking not found" message; no data shown.	Not executed	Not run
TC-16	4	Cancel eligible booking	Confirmed, eligible booking	1. Open booking. 2. Choose cancel. 3. Confirm.	Booking and ticket become CANCELLED together; availability restored; success message.	Not executed	Not run
TC-17	4	Cancel already cancelled booking	Cancelled booking exists	1. Attempt cancellation again.	"Already cancelled" message; no data change; availability not restored twice.	Not executed	Not run
TC-18	5	Administrator adds a flight	Logged in as ADMIN	1. Open add-flight form. 2. Enter valid data. 3. Submit.	Flight saved ACTIVE; appears in search results.	Not executed	Not run
TC-19	5	Administrator updates a flight	Existing flight	1. Edit fare or schedule. 2. Save.	Changes saved and shown in search; existing bookings keep stored fare.	Not executed	Not run
TC-20	5	Invalid fare or schedule entry	Logged in as ADMIN	1. Enter negative fare or arrival before departure. 2. Submit.	Validation error; record unchanged.	Not executed	Not run
TC-21	3	Database failure during booking	Ability to force a failure (e.g. disconnect after first insert)	1. Start a booking. 2. Force failure mid-transaction.	Transaction rolled back; no partial rows; safe error message.	Not executed	Not run
TC-22	4	Unauthorized access to another passenger's booking	Two passenger accounts; booking owned by user A	1. Log in as user B. 2. Enter user A's PNR or URL.	Access denied or not found; no data of user A shown.	Not executed	Not run

9.1 Security Testing

Table 23. Security test cases (all unexecuted)

Test ID	Module	Test scenario	Preconditions	Test steps	Expected result	Actual result	Status
ST-01	1	SQL injection attempt	Login and search forms available	1. Enter ' OR '1'='1 and similar strings in login and search fields.	Input treated as data; no authentication bypass; no SQL error exposed.	Not executed	Not run
ST-02	5	Unauthorized administrative endpoints	Passenger and anonymous clients	1. Request each admin URL while anonymous and as passenger.	Redirect to login or access denied for all.	Not executed	Not run
ST-03	1	Session invalidation after logout	Logged-in user; copy of session cookie	1. Log out. 2. Replay the old cookie on a protected URL.	Old session rejected.	Not executed	Not run
ST-04	1	Expired session handling	Short timeout configured for test	1. Log in. 2. Wait beyond timeout. 3. Request protected page.	Redirect to login with message.	Not executed	Not run
ST-05	1	Input-validation bypass	Request tool available	1. Submit forms with client checks disabled and with oversized, malformed, or script-containing values.	Server rejects or encodes input; no stored script runs.	Not executed	Not run
ST-06	4	Unauthorized booking lookup	Two accounts	1. As user B, change PNR or id parameters to user A's values.	No data of user A returned.	Not executed	Not run
ST-07	3	Safe handling of database errors	Database stopped or misconfigured in test	1. Perform search and login.	Controlled error page; no stack trace, SQL, or credentials.	Not executed	Not run

9.2 Acceptance Testing

Acceptance testing shall be performed by the team and demonstrated to the mentor. The team prepares a clean database with seed data (one administrator, at least two passengers, and several active flights), deploys the application to Tomcat 10.1, and executes the cases in this section in the order of the requirements-to-acceptance matrix in Section 10. For each case the tester records the actual result, status, date, and tester name. Defects are logged with the failing requirement identifier, fixed, and retested; regression cases are re-run after each fix. The mentor may select additional scenarios during the demonstration.

10. Acceptance Criteria

The matrix below links each acceptance criterion to the requirements and test cases that verify it. Every criterion is Proposed — Pending Verification until the listed tests have been executed with recorded results.

Table 24. Requirements-to-acceptance matrix

No.	Acceptance criterion	Requirements	Tests	Status
AC-01	A passenger can register using valid information.	FR-01, FR-04, FR-08	TC-01, TC-02	Proposed — Pending Verification
AC-02	Invalid credentials do not grant access.	FR-02	TC-04, ST-01	Proposed — Pending Verification
AC-03	Passenger accounts cannot perform administrator-only actions.	FR-05, FR-41	TC-06, ST-02	Proposed — Pending Verification
AC-04	Valid search criteria produce matching flights when matching data exists.	FR-09, FR-10, FR-13	TC-07, TC-08, TC-09	Proposed — Pending Verification
AC-05	A successful booking produces a unique PNR.	FR-16, FR-17	TC-10, TC-12	Proposed — Pending Verification
AC-06	Booking and passenger information are stored consistently.	FR-19, FR-20	TC-10, TC-13	Proposed — Pending Verification
AC-07	Authorized users can retrieve permitted booking and ticket details; others cannot.	FR-24 to FR-27	TC-14, TC-15, TC-22, ST-06	Proposed — Pending Verification
AC-08	Eligible cancellations update the appropriate statuses; repeated cancellation changes nothing.	FR-28 to FR-32	TC-16, TC-17	Proposed — Pending Verification

No.	Acceptance criterion	Requirements	Tests	Status
AC-09	Administrators can manage flight information according to their permissions, with validation.	FR-35 to FR-40	TC-18, TC-19, TC-20	Proposed — Pending Verification
AC-10	Database errors do not leave partially committed booking records.	FR-20, FR-23	TC-21, ST-07	Proposed — Pending Verification
AC-11	The session ends at logout and after expiry.	FR-03, FR-07	TC-05, ST-03, ST-04	Proposed — Pending Verification
AC-12	The application builds and starts in the documented environment when configured correctly.	Sections 2.4, 3.2	Deployment check	Proposed — Pending Verification

A successful compilation alone does not establish that these criteria are met.

11. Assumptions, Dependencies and Constraints

11.1 Assumptions

The following are reasonable assumptions, not confirmed facts:

- Development and testing take place in a local environment with a locally installed MySQL server.
- The database is available whenever the application runs.
- Users access the application through a supported modern browser.
- Users provide accurate data, which the system validates but cannot verify against external sources.
- Flight data is entered and maintained by administrators.
- Administrator accounts are provisioned by the team (for example by seed data), not through public registration.
- Booking references (PNR) are application-level references and are not issued by any airline system.

11.2 Dependencies

- Java runtime (JDK 21).
- Servlet container (Apache Tomcat 10.1 with Jakarta Servlet 6.0).
- MySQL server and the MySQL JDBC driver.
- Apache Maven for building and dependency resolution.
- Browser compatibility with the HTML, CSS, and JavaScript used.
- Configuration files holding connection settings and application parameters.

11.3 Constraints

Confirmed project constraints:

- The project period is 6 July 2026 to 6 October 2026, within the 2026–2027 academic session.
- The work is carried out by a team of five students, with one module per member.
- The stack is Java, JSP/Servlets, JDBC, and MySQL as specified in Section 2.4.

Constraints inherited from scope and resources (to be confirmed):

- Available development hardware is limited to student machines.
- Integration with external systems is limited or absent, as stated in Section 1.2.
- Data is local test data and not live airline data.
- Testing and deployment are limited to a local or college environment and do not cover production loads.

12. Future Enhancements

The following are possible enhancements. None is a current feature.

Table 25. Possible future enhancements

Enhancement	Potential benefit	Major dependency
Secure payment-gateway integration	Allows paid bookings and a realistic reservation flow.	Gateway account, PCI-aware design, HTTPS, refund policy.
Email or SMS notifications	Confirms bookings and cancellations to passengers.	Mail or SMS provider and credentials.
PDF ticket generation	Gives printable, shareable tickets.	PDF library; ticket template.
Seat-map selection	Lets passengers choose specific seats.	Seat table and aircraft layout data.
Password reset	Lets users recover accounts.	Email delivery; secure token handling.
Multi-factor authentication	Strengthens account security, especially for administrators.	Second-factor mechanism (for example one-time codes).
Google Sign-In (if configured and verified)	Simplifies login.	OAuth client credentials and consent configuration.
Live airline inventory integration	Replaces local availability with real data.	Airline or distribution API agreements.
Flight-status APIs	Shows delays and cancellations.	External data provider.
Refund management	Completes the cancellation process.	Payment integration and a defined refund policy.
Enhanced reporting and analytics	Gives administrators deeper insight.	Additional queries or a reporting library.

Enhancement	Potential benefit	Major dependency
Audit logging	Records who changed what and when.	Audit table and logging design.
Automated backups	Reduces data-loss risk.	Scheduler and storage location.
Secure cloud deployment	Makes the system reachable and robust.	Hosting account, HTTPS certificate, managed secrets.
Accessibility improvements	Broadens usability.	Accessibility audit and testing tools.
Multilingual support	Serves users of several languages.	Resource bundles and translations.

13. Conclusion

This Software Requirements Specification defines the purpose, scope, users, requirements, architecture, data design, and verification approach for SkyTech Airlines – Airline Reservation and Passenger Management System. It sets out 42 functional requirements, a set of verifiable non-functional requirements, seven proposed diagrams, a proposed relational schema, and a set of test cases and acceptance criteria.

The system is organized into five modules: authentication and user management, flight search and availability, booking and passenger management, ticketing and cancellation, and administrative flight management. Authentication supplies identity and role; search supplies valid flights; booking converts a selected flight and passenger details into a reservation with a unique PNR; ticketing and cancellation maintain the lifecycle of that reservation; and administration maintains the flight data on which everything depends. Because reservation data and personal information are involved, security controls and database integrity rules are treated as core requirements, not additions.

The document is a guide for development, integration, and evaluation. It describes intended behaviour; it does not assert that any function has been implemented, tested, or deployed. The proposed diagrams and schema shall be checked against the repository and the actual SQL schema, and test results shall be recorded only from executed tests.

14. References

The following official documentation was identified as relevant. Dates of access and any edition-specific details are to be added when the team verifies each source.

[1] Oracle, "Java Documentation." [Online]. Available: https://docs.oracle.com/en/java/

[2] Eclipse Foundation, "Jakarta Servlet Specification." [Online]. Available: https://jakarta.ee/specifications/servlet/

[3] Apache Software Foundation, "Apache Tomcat Documentation." [Online]. Available: https://tomcat.apache.org/

[4] Oracle, "Java Database Connectivity (JDBC) Documentation." [Online]. Available: https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/

[5] Eclipse Foundation, "Jakarta Pages (JSP) Specification." [Online]. Available: https://jakarta.ee/specifications/pages/

[6] Oracle, "MySQL Documentation." [Online]. Available: https://dev.mysql.com/doc/

[7] Apache Software Foundation, "Apache Maven Guides." [Online]. Available: https://maven.apache.org/guides/

[8] OWASP Foundation, "OWASP Cheat Sheet Series." [Online]. Available: https://cheatsheetseries.owasp.org/

[9] Object Management Group, "Unified Modeling Language (UML) Specification." [Online]. Available: https://www.omg.org/spec/UML/